

Big Data: Management and Analytics

Task 1

In this task we will get familiar with the MapReduce programming model. For this we will use Python again. Recall the previous assignments, in which Python was introduced.

Suppose you are given the matrices $A \in \mathbb{R}^{i \times j}$ and $B \in \mathbb{R}^{j \times k}$

Let there be $A := \begin{pmatrix} 11 & 19 & 5 \\ 13 & 7 & 2 \end{pmatrix}$

and $B := \begin{pmatrix} 3 & 17 \\ 29 & 37 \\ 31 & 23 \end{pmatrix}$

(a) State how a matrix multiplication is done in general. Multiply both matrices with each other and write down the resulting matrix C. Could you achieve other solutions for C with the matrices A and B?

Matrix multiplication in general – mathematical form of multiplying matrix is

step 1: Multiply the first row of matrix A with first column of matrix B and add the values

step 2: Multiply the first row of matrix A with second column of matrix B and add the values

step 3: Multiply the second row of matrix A with first column of matrix B and add the values

step 4: Multiply the second row of matrix A with second column of matrix B and add the values

and the resultant values is assigned in the matrix C.

Let there be $A := \begin{pmatrix} 11 & 19 & 5 \\ 13 & 7 & 2 \end{pmatrix}$

and $B := \begin{pmatrix} 3 & 17 \\ 29 & 37 \\ 31 & 23 \end{pmatrix}$

$$A \times B = C$$

$$= 11 * 3 + 19 * 29 + 5 * 31 = 739$$

$$= 11 * 17 + 19 * 37 + 5 * 23 = 1005$$

$$= 13 * 3 + 7 * 29 + 2 * 31 = 304$$

$$= 13 * 17 + 7 * 37 + 2 * 23 = 526$$

$$C = \begin{pmatrix} 739 & 1005 \\ 304 & 526 \end{pmatrix}$$

$$A \times B = C$$

$$\begin{pmatrix} 11 & 19 & 5 \\ 13 & 7 & 2 \end{pmatrix} \times \begin{pmatrix} 3 & 17 \\ 29 & 37 \\ 31 & 23 \end{pmatrix} = \begin{pmatrix} 739 & 1005 \\ 304 & 526 \end{pmatrix}$$

. Could you achieve other solutions for C with the matrices A and B?

for achieving other solutions for C with matrices A and B python program can be used.

```

matrix.py - C:/Users/asus/OneDrive/Documents/Python Scripts/Assignment3/matrix.py (3.9.12)
File Edit Format Run Options Window Help
# Program to multiply two matrices using nested loops

# 2x3 matrix
A = [ [11,19,5], [13,7,2]]
# 3x2 matrix
B = [ [3,17], [29,37], [31,23] ]

# resultant matrix
result_C = [[0,0],[0,0]]

# iterating rows of A matrix
for i in range( len(A) ):
    # iterating columns of B matrix
    for j in range(len(B[0])):
        # iterating rows of B matrix
        for k in range(len(B)):
            result_C[i][j] += A[i][k] * B[k][j]

print("The resultant matrix is :\t ")
for r in result_C:
    print(r)

///
= RESTART: C:/Users/asus/OneDrive/Documents/Python Scripts/Assignment3/matrix.py
The resultant matrix is
[739, 1005]
[304, 526]
>>> |

```

Here using python, we got the same result. As done with general method.

(b) Explain how the matrices can be multiplied by MapReduce. Begin with rewriting the matrices as tuples (i, j, a_{ij}) and (j, k, b_{jk}). Explain all steps in general and give at least one numerical example for each step. Verify the results with your solution in (a).

Map reduce is a technique in big data which help us to sub divide large program into small task and run task parallel to make computation faster. It has two parts Mapper and reducer

Mapper: takes raw data as input and organizes into key value pair. Key value pair contains a key and a value.

Reducer: It is the function that process data in parallel and produce final output.

Let us consider the matrix multiplication example to visualize MapReduce. Consider the following matrix:

$$A := \begin{pmatrix} 11 & 19 & 5 \\ 13 & 7 & 2 \end{pmatrix}$$

$$B := \begin{pmatrix} 3 & 17 \\ 29 & 37 \\ 31 & 23 \end{pmatrix}$$

Here matrix A is a 2×3 matrix which means the number of rows(i)=2 and the number of columns(j)=3. Matrix B is also a 3×2 matrix where number of rows(j)=3 and number of columns(k)=2.

Each cell of the matrix is labelled as A_{ij} and B_{jk}. Example: A₁₁ = 11, B₁₁ = 3 i.e. The position at row (1) and column (1) in matrix A the value is 11, Likewise the position at row (1) and column (1) in matrix B the value is 3.

Now matrix multiplication using map reduce has 1 mapper and 1 reducer. The Formula is:

Mapper for Matrix A (k, v)=(i, k), (A, j, A_{ij})) for all k

Mapper for Matrix B (k, v)=(i, k), (B, j, B_{jk})) for all i

Therefore computing the mapper for Matrix A:

Matrix A (k, v)=(i, k), (A, j, A_{ij})) for all k

k, i, j computes the number of times it occurs.

Here k has two values 1 and 2 as there is two columns in matrix B.

The i can have values 1 and 2 as it represents rows in matrix A.

The j can have values 1,2, and 3 as it represents columns in matrix A

Thus, substituting all values in formula

Matrix A (k, v)=(i, k), (A, j, A_{ij})) for all k

For k =1, i = 1

((1,1), (A,1,11)) # here (1,1) is the key pair and (A,1,1) is the value pair.
((1,1), (A,2,19))
((1,1), (A,3,5))

i= 2

$((2,1), (A,1,13))$
 $((2,1), (A,2,7))$
 $((2,1), (A,3,2))$

For k =2 ,i = 1

$((1,2), (A,1,11))$
 $((1,2), (A,2,19))$
 $((1,2), (A,3,5))$

i= 2

$((2,2), (A,1,13))$
 $((2,2), (A,2,7))$
 $((2,2), (A,3,2))$

Computing the mapper for Matrix B

Mapper for Matrix B (k, v)=((i, k), (B, j, Bjk)) for all i

i, j, k computes the number of times it occurs.

Here i has three values 1, 2 and 3 as there is three columns in matrix A.

The j can have values 1, 2 and 3 as it represents rows in matrix B.

The k can have values 1,2 as it represents columns in matrix B.

Matrix B (k, v)=((i, k), (B, j, Bjk)) for all i

For i =1

$((1,1), (B,1,3))$
 $((1,2), (B,1,17))$
 $((1,1), (B,2,29))$
 $((1,2), (B,2,37))$
 $((1,1), (B,3,31))$
 $((1,2), (B,3,23))$

i= 2

$((2,1), (B,1,3))$
 $((2,2), (B,1,17))$
 $((2,1), (B,2,29))$
 $((2,2), (B,2,37))$
 $((2,1), (B,3,31))$
 $((2,2), (B,3,23))$

i=3

$((3,1), (B,1,3))$
 $((3,2), (B,1,17))$
 $((3,1), (B,2,29))$
 $((3,2), (B,2,37))$
 $((3,1), (B,3,31))$
 $((3,2), (B,3,23))$

Here we have mapped matrix A and matrix B now next step in map reduce is to reduce () the matrix A and B

The formula for Reducer is:

Reducer(k, v)=(i, k)=>Make sorted Alist and Blist

(i, k) => Summation (Aij * Bjk) for j

Output =>((i, k), sum)

We can observe from Mapper computation that 4 pairs are common

(1, 1), (1, 2), (2, 1) and (2, 2)

With this common pair we make a list separate for matrix A that is Alist and for matrix B that is Blist and apply the reducer formula.

As (3,1) (3,2) are not common in both Matrix A and B it is excluded.

Applying Reducer Formula

Reducer(k, v)=(i, k)=>Make sorted Alist and Blist

(i, k) => Summation (Aij * Bjk) for j

Output =>((i, k), sum)

First we compute on the pair (1,1)

(1, 1) =>Alist = {(A, 1, 11), (A, 2, 19), (A,3,5)}

Blist = {(B, 1, 3), (B, 2, 29), (B,3,31)}

Now Aij x Bjk: [(11*3) + (19*29) +(5*31)] =739 ----- (i)

(1, 2) =>Alist = {(A, 1, 11), (A, 2, 19), (A,3,5)}

Blist = {(B, 1, 17), (B, 2, 37), (B,3,23)}

Now Aij x Bjk: [(11*17) + (19*37) +(5*23)] =1005----- (ii)

(2,1) =>Alist = {(A, 1, 13), (A, 2, 7), (A,3,2)}

Blist = {(B, 1, 3), (B, 2, 29), (B,3,31)}

Now Aij x Bjk: [(13*3) + (7*29) +(2*31)] =304 -----(iii)

(2,2) =>Alist = {(A, 1, 13), (A, 2, 7),(A,3,2)}

Blist = {(B, 1, 17), (B, 2, 37),(B,3,23)}

Now Aij x Bjk: [(13*17) + (7*37) +(2*23)] =526 ----- (iv)

From (i), (ii), (iii) and (iv) we conclude that

((1, 1), 739)

((1, 2), 1005)

((2, 1), 304)

((2, 2), 526)

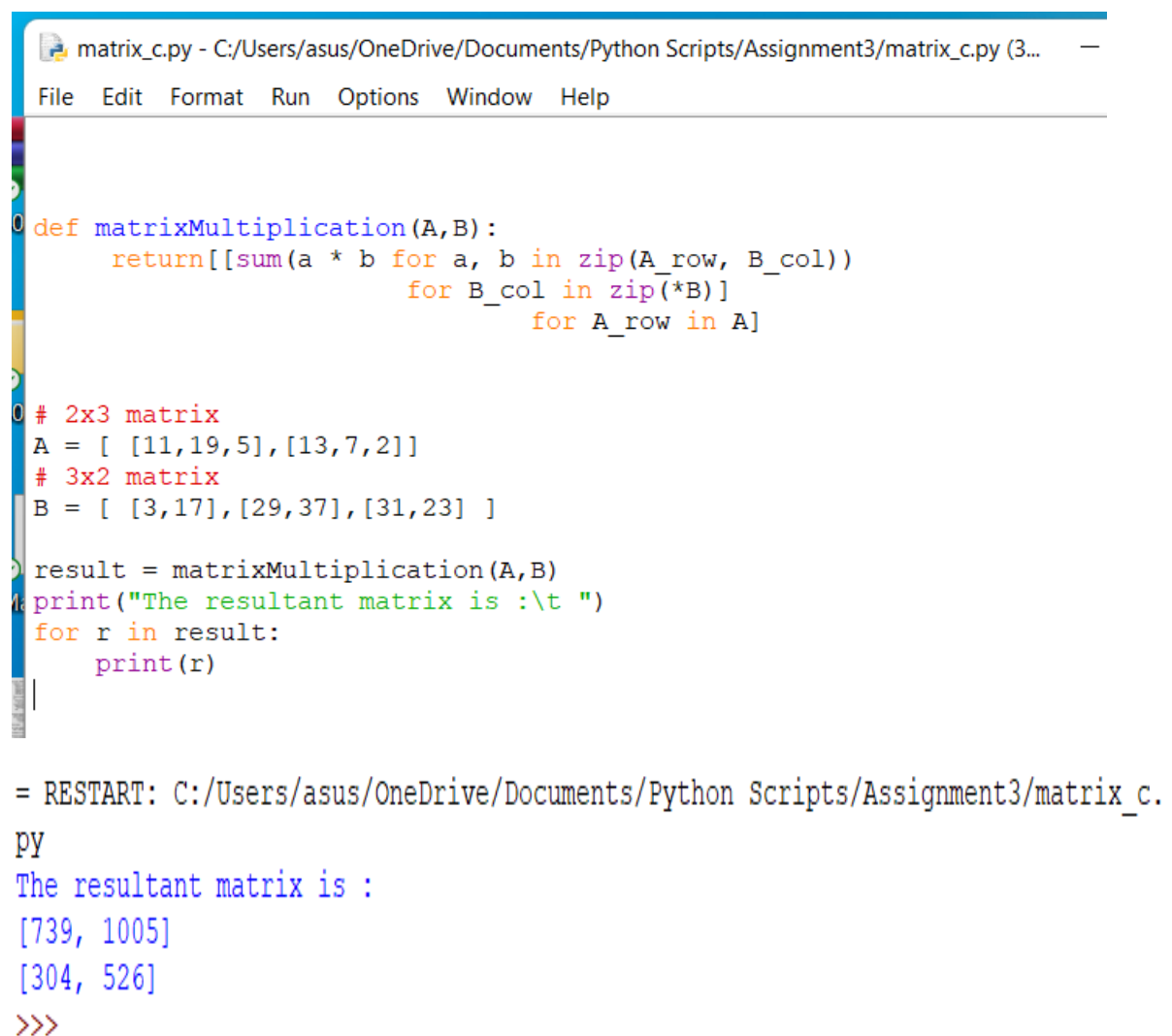
Therefore the Final Matrix is:

$$C = \begin{pmatrix} 739 & 1005 \\ 304 & 526 \end{pmatrix}$$

$$\begin{pmatrix} 11 & 19 & 5 \\ 13 & 7 & 2 \end{pmatrix} \times \begin{pmatrix} 3 & 17 \\ 29 & 37 \\ 31 & 23 \end{pmatrix} = \begin{pmatrix} 739 & 1005 \\ 304 & 526 \end{pmatrix}$$

From the solution it is clear that the result of solution(a) is same result as solution (b).

(c) Implement a function matrixMultiplication() with python, which takes 2 matrices as arguments. The function returns the product of both matrices. Within the function use the type of functions, which were introduced in Assignment 1 Task 2 !! Again, verify your result with the solutions from (a) and (b).



```
matrix_c.py - C:/Users/asus/OneDrive/Documents/Python Scripts/Assignment3/matrix_c.py (3...
File Edit Format Run Options Window Help

def matrixMultiplication(A,B):
    return[[sum(a * b for a, b in zip(A_row, B_col))
            for B_col in zip(*B)]
           for A_row in A]

# 2x3 matrix
A = [ [11,19,5],[13,7,2]]
# 3x2 matrix
B = [ [3,17],[29,37],[31,23] ]

result = matrixMultiplication(A,B)
print("The resultant matrix is :\t ")
for r in result:
    print(r)

= RESTART: C:/Users/asus/OneDrive/Documents/Python Scripts/Assignment3/matrix_c.
py
The resultant matrix is :
[739, 1005]
[304, 526]
>>>
```

In solution (a) and (b) we have achieved the same result for the matrix multiplication.

In solution (a) matrix multiplication was done using general mathematical formula and python solution where we achieved the same result as above for matrix $A \times B = C$.

In solution (b) we have done map reduce using the same matrix A and B and achieved the same result.

In solution (c) we have done matrix multiplication using user defined function and used list comprehension. Here also we have achieved the same result as in solution (a) and (b).

TASK 2

a) Given a list a = ["I", "like", "cookies"] and another list b = a. Replace in the list b the element "cookies" with "apples". Finally print both lists (a and b). What do you observe? What leads to the observed behavior?

```
In [25]: a = ["I", "like", "cookies"]
          b=a
          b[2]='apples'
          print(a)
          print(b)

['I', 'like', 'apples']
['I', 'like', 'apples']
```

When the list a is assigned to new list b, and the change done in the new list b alters the list a, from this observation, we learned that in python list a and b are pointing to same object. In python everything is object and when list a is assigned to list b they are pointing to same object, so modifying b will automatically modify a. In python = operator does not create a new object instead it only creates a new variable that shares the reference of the original object.

(b) Given a list a = ["I", "like", "cookies"] and another list which takes a shallow copy of a: b = a[:]. Like in the previous task (a), replace in the list b the element "cookies" with "apples". Finally print both lists (a and b). What do you observe now?

```
In [3]: a = ['I','like','cookies']
          b = a[:]
          b[2] = 'apples'
          print("list a: ",a)
          print("list b: ",b)
          print(id(a),id(b))

list a:  ['I', 'like', 'cookies']
list b:  ['I', 'like', 'apples']
2108499923456 2108500126144
```

When list a is assigned to list b as shallow copy in statement 2. And list b is altered at index 2, From the result we can observe that list a has not changed while list b has changed. This is because list b was assigned as shallow copy, in python shallow copy creates a new object

which stores the reference of the source object. So, the change happened to the new object does not affect the source object.

(c) Now we are given a list a = ["I", "like", ["chocolate", "cookies"]] and another list which takes a shallow copy of a: b = a[:]. Change now the element "cookies" with "apples" in b. Now print again both lists (a and b). What do you observe? What leads to the observed behavior?

```
In [7]: a = ['I', 'like', ['chocolate', 'cookies']]
        b = a[:]
        b[2][1] = 'apples'
        print("list a: ",a)
        print("list b: ",b)
        print(id(a),id(b))
        print(id(a[2]),id(b[2]))

list a:  ['I', 'like', ['chocolate', 'apples']]
list b:  ['I', 'like', ['chocolate', 'apples']]
2108500354560 2108500134528
2108500354624 2108500354624
```

Here a is the nested list and b is the shallow copy of nested list a. And the list b is altered and both the list are printed here we can observe that list a is also altered along with list b. from this we can understand that shallow copy only creates the outer list copy and not the nested list copy. Nested list object is same for both list a and b. If you try to modify the nested list then it will modify the original list too as the nested list object is same for both lists.

d) Like in the previous task, we are given a list a = ["I", "like", ["chocolate", "cookies"]]. Another list b = deepcopy(a) takes a deep copy from a. Now change the element "cookies" with "apples" in b. Print both lists (a and b). What do you observe now?

Hint: For deep copy, first type: from copy import deepcopy in the head of your python-file.


```
In [11]: from copy import deepcopy

a = ['I','like',['chocolate', 'cookies']]
b = deepcopy(a)
b[2][1] = 'apples'
print("list a: ",a)
print("list b: ",b)
print(id(a[2]),id(b[2]))

list a: ['I', 'like', ['chocolate', 'cookies']]
list b: ['I', 'like', ['chocolate', 'apples']]
2108500349632 2108471832384
```

Here a is a nested list and b is the deep copy of the nested list and the list b is altered at index [2][1], From the result we can observe that b is the deep copy of a that is it is a cloned list, here the change in the cloned list would not affect the original list. So, the original list is not changed but the list b is changed at the index [2][1].

(e) Create a dictionary with n entries, where the keys are enumerated from 0 to n – 1 and the values are their corresponding keys squared. Use list comprehensions! Example for the expected result: n = 7: {0:0, 1:1, 2:4, 3:9, 4:16, 5:25, 6:36}

```
In [24]: d1 = {x : x**2 for x in range(7)}
print("Dictionary is: ",d1)
#Using zip
keys = range(7)
values = [x**2 for x in keys]
print(list(zip(keys, values)))
d1 = dict(zip(keys, values))
print(d1)

Dictionary is: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36)]
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
```

(f) Use the dictionary from (e). Write a list comprehension to get a list of all the keys of the dictionary.

```
In [20]: #it actually corresponds to d.keys()
dlis = [x for x in d1]
print(dlis)

[0, 1, 2, 3, 4, 5, 6]
```